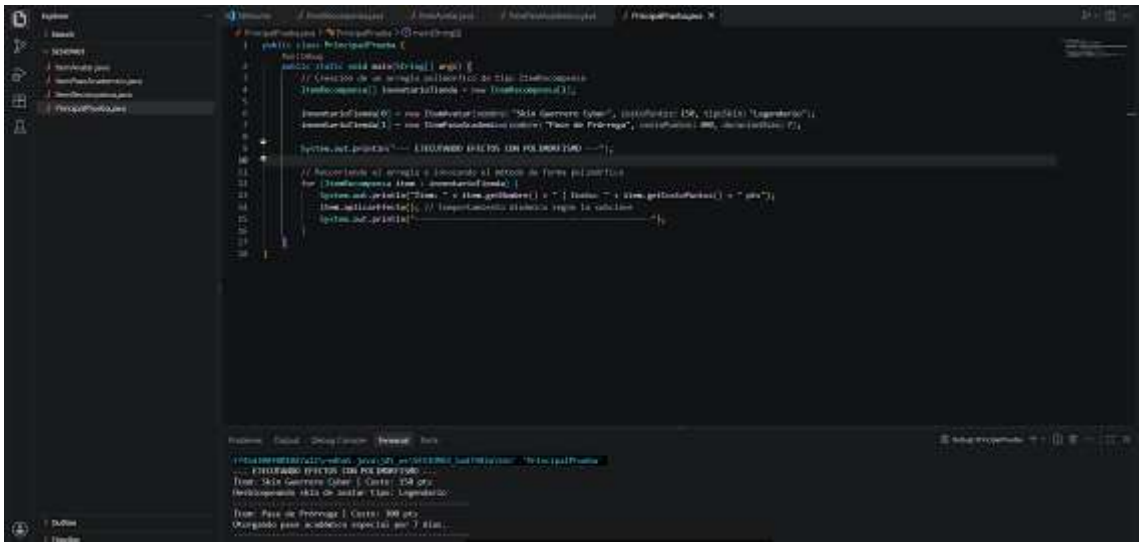
Guía de Aprendizaje N° 3: Polimorfismo y Clases Abstractas en Java





En el ejercicio, la clase ItemRecompensa es abstracta, y eso significa que no puedo hacer algo como new ItemRecompensa(...) directamente. La razón es que esa clase está "incompleta": tiene el método aplicarEfecto() declarado pero sin cuerpo, o sea, sin decir qué hace. Java no me deja crear un objeto de una clase así porque sería como querer usar un formulario en blanco en lugar de una copia ya llenada — le falta información. Por eso estoy obligado a crear una clase hija concreta, como ItemAvatar o ItemPaseAcademico, y ahí sí definir qué hace aplicarEfecto() en cada caso. La ventaja de esto es que la clase madre me obliga a que todas las recompensas tengan ese método, pero cada tipo de recompensa decide cómo lo implementa. Gracias a eso funciona el polimorfismo: puedo recorrer un arreglo de ItemRecompensa y llamar aplicarEfecto() en cada elemento, y cada objeto responde a su manera según lo que realmente es.

Sobre @Override, al principio pensaba que era solo decorativo, pero en realidad es una ayuda del compilador. Lo que hace es avisarme si el método que estoy escribiendo

en la subclase realmente está sobrescribiendo el método del padre. Si me equivoco en el nombre (por ejemplo escribo `aplicarEfectos()` con "s" al final) o en los parámetros, el compilador me marca error en vez de dejarme crear un método nuevo sin darme cuenta. Eso me salvó varias veces porque, sin esa anotación, el código compilaría igual pero el polimorfismo no funcionaría y no sabría por qué. En resumen, `@Override` es como un "check" que me confirma que voy por buen camino cuando sobrescribo un método